

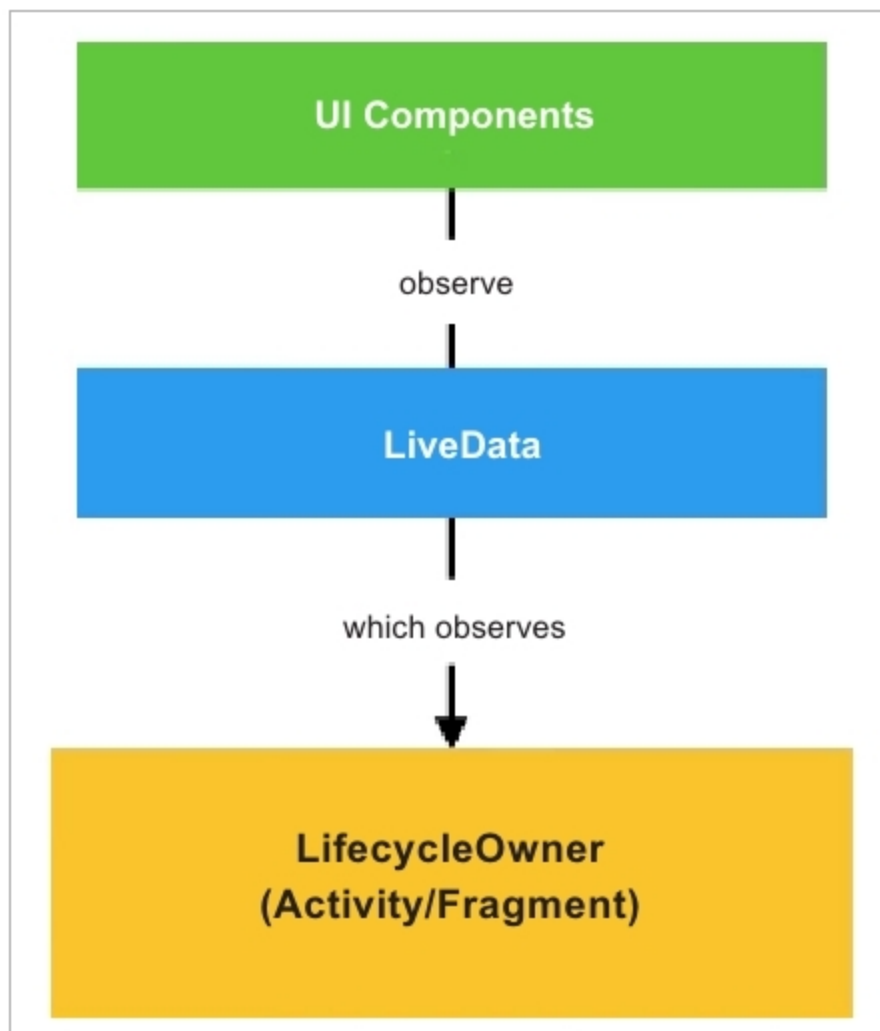


Figure 2: Flow of LiveData

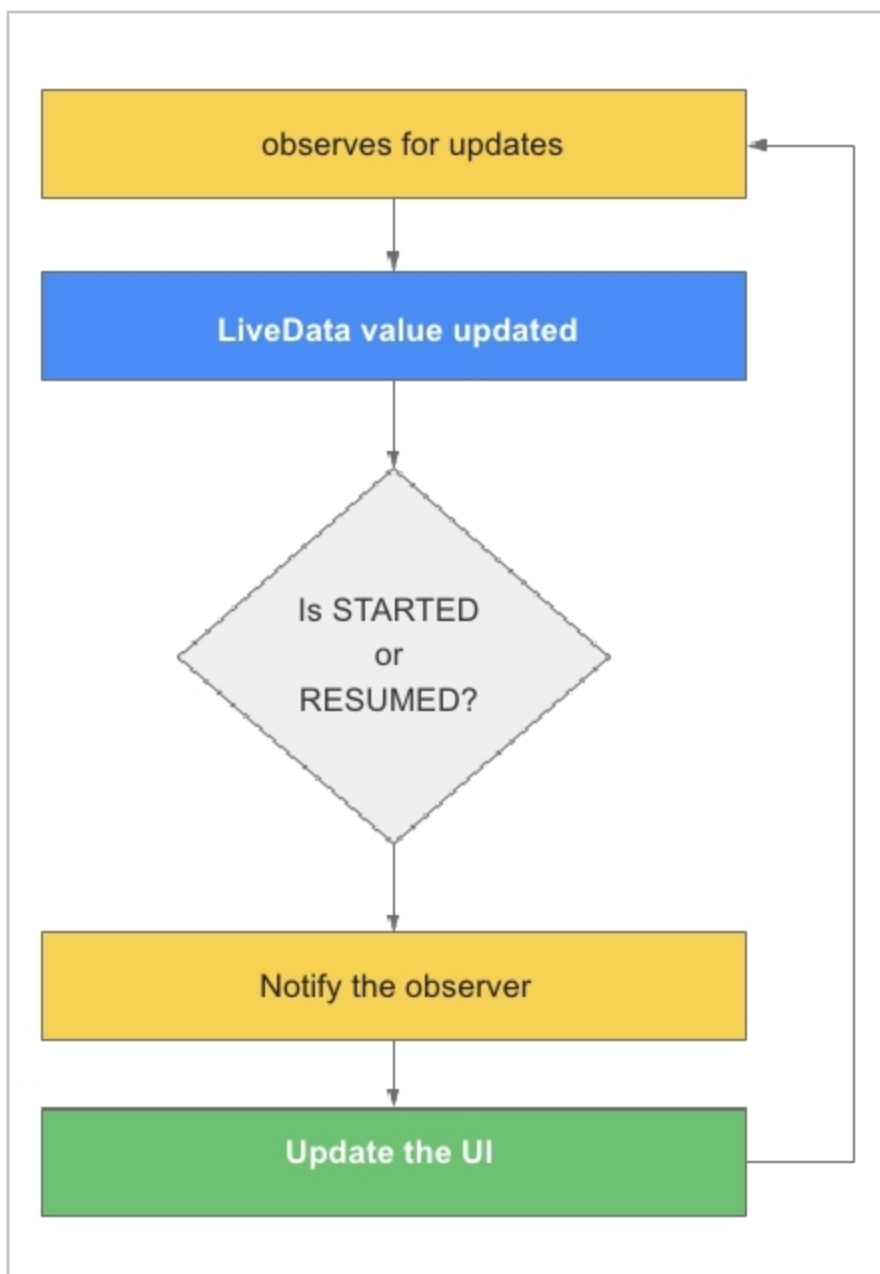


Figure 3: Working of LiveData

There is more to this flow but, for the sake of simplicity, I have focused only on the important parts in Figure 3.

Listed below are the key advantages of LiveData:

- Always updates data
- It is life cycle-aware
- Handles proper configuration changes
- Has no memory leaks

ViewModel

ViewModel is a container to store all your UI related data. It contains data required by the UI. For example, if the UI is a list of news items, then ViewModel will contain/store news and provide it to the UI when required.

This enables a separation of concerns, and the UI controller (Activity and Fragment) contains purely UI related code like *findViewById*, click listeners, manipulating widgets, etc, while the ViewModel contains data which is to be shown on the UI.

This decoupling solves a major problem which Android developers face, i.e., the app crashing or data getting cleared on rotating the device. In a nutshell, ViewModels survive configuration changes.

An example: Let's say you want to build a UI showing a list of news articles. Here, the data required is a list of news articles. Hence our 'ViewModel' will look something like what's shown below:

```

class NewsViewModel() : ViewModel() {

    private var newsArticles: List<NewsArticles>

    init {
        // Load the data over here
        newsArticles = ....
    }

    fun getNewsArticles(): List<NewsArticles> {
        return newsArticles
    }
}
  
```

The *NewsViewModel* is holding the data, which in this case is 'newsArticles', and is fetching it from somewhere (ideally it is delegated to a repository, which we will look at later). As discussed earlier, ViewModel will usually hold the data in the form of LiveData to update the UI immediately when data changes.

```

class NewsViewModel() : ViewModel() {

    private var newsArticles: LiveData<List<NewsArticles>>

    init {
        // Load the data over here
  
```